

Anchor Parsing

May 2026

Abstract We present a parser that given a context free grammar $G = (N, \Sigma, P, S)$ and a string β such that $\exists x, y \in \Sigma^* : x\beta y \in L(G)$, enumerates all possible parse trees that can cover β with the missing left and right context, if any. We can use this to extract parse trees from incomplete code, such as fragments found in git patches.

1 Introduction

Git patches only record the added and removed lines in a codebase, presenting the code as fragments with minimal surrounding context. When studying how code changes across a series of patches, there is a need to understand what the changed code is syntactically: a declaration, a function call, a statement, etc.

```
int size, rc = 0;

while (n > 0) {
- size = zpci_get_max_write_size((u64 __force)
src,
- (u64) dst, n,
- ZPCI_MAX_READ_SIZE);
+ size = zpci_get_max_io_size((u64 __force) src,
+ (u64) dst, n,
+ ZPCI_MAX_READ_SIZE);
rc = zpci_read_single(dst, src, size);
if (rc)
break;
```

A fragment from a Linux kernel patch.

```
int size, rc = 0;

while (n > 0) {
size = zpci_get_max_io_size((u64 __force) src,
(u64) dst, n,
ZPCI_MAX_READ_SIZE);
rc = zpci_read_single(dst, src, size);
if (rc)
break;
```

Code to be parsed (highlighted) with surrounding context.

Parsing this input is complicated, since standard parsers assume a well-formed, complete input. When presented with a fragment they fail at the first token that cannot extend a valid parse due to the missing surrounding context. Parsers with error recovery strategies (tree-sitter and similar tools) salvage a single best parse but suppress ambiguity that might exist.

To resolve these issues, we change the basic assumptions.

1. We assume that the input string belongs to the language and has a valid derivation
2. We only need the smallest non terminals that can cover the entire input.

Given a context-free grammar G and a token sequence β , our algorithm returns the covering set $\mathcal{C}(\beta) \subseteq N$ of nonterminals under which β appears as a substring, together with parse trees rooted at nonterminals in $\mathcal{C}(\beta)$ and descriptions of the missing context. This allows us to compare two fragments with the same covering set directly as instances of the same grammatical construct.

In this paper we describe an adaptation of the h-cover framework of [1] to the fragment parsing problem, returning $\mathcal{C}(\beta)$ with parse trees and context descriptions for any context-free grammar. We evaluate it on real fragments drawn from Linux kernel patches.

Bibliography

- [1] G. Satta and O. Stock, “BiDirectional Context-Free Grammar Parsing for Natural Language Processing,” *Artificial Intelligence*, vol. 69, pp. 123–164, 1994.